CSE 8803: CMC

Assignment 2

1. Download and install the network visualization software Gephi from gephi.org.

Gephi tutorial here and here.

Download a network dataset (e.g. you can find a few here). Make a visualization in Gephi.

2. Six degrees of separation is the phrase commonly used to describe the interconnected nature of human acquaintances. Various studies have shown that any random pair of people in the world can be connected to one another by a short chain of people (typically around six), each of whom knows the next fairly well. If we represent people as nodes and acquaintanceships as neighbors, we reduce the problem to the study of a relationship network.

Many interesting problems arise from studying properties of randomly generated networks. A random network is constructed probabilistically according to some definite rules; studying such a random network usually is done by studying the entire ensemble of networks, each weighted by the probability that it was constructed.

Here, we will generate some random networks, and calculate the distribution of distances between pairs of points. We will study small world networks, a theoretical model that suggests how a small number of shortcuts (unusual international and intercultural friendships) can dramatically shorten the typical chain lengths.

(a) Constructing a small world network. The L nodes in a small world network are arranged around a circle. There are two kinds of edges. Each node has Z short edges connecting it to its nearest neighbors around the circle (up to a distance $Z/2$). In addition, there are $p \times L \times Z/2$ shortcuts added to the network, which connect nodes at random (see Figure 1).

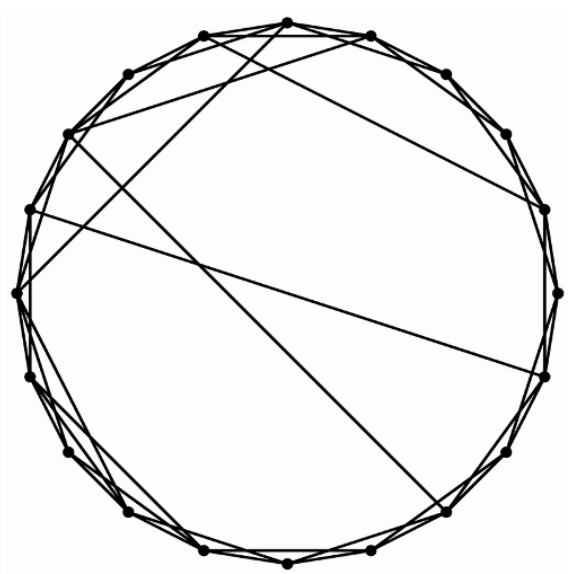

Figure 1

Python code fragments are provided to you with this assignment. The code defines a network class

with several functions. Complete the code by completing those functions:

i **HasNode (node)**, which checks to see if a node is already in the network.

ii **AddNode(node)**, which adds a new node to the network (if it is not already there).

iii **AddEdge(node1, node2)**, which adds a new edge to the network.

iv **GetNodes()**, which returns a list of existing nodes.

v **GetNeighbors(node)**, which returns the neighbors of an existing node.

Write a function to construct a small world network, which (given L, Z, and p) adds the nodes and the short edges, and then randomly adds the shortcuts. Use the code provided to draw this small world graph, and check that you have implemented periodic boundary conditions correctly (each node i should be connected to nodes $(i - Z/2) \bmod L, \ldots, (i + Z/2) \bmod L) \cdot$.

(b) Measuring the minimum distances between nodes. The most studied property of small world graphs is the distribution of shortest paths between nodes. Without the long edges, the shortest path between i and j will be given by hopping in steps of length $Z/2$ along the shorter of the two arcs around the circle; there will be no paths of length longer than L/Z (halfway around the circle), and the distribution $\rho(\ell)$ of path lengths ℓ will be constant for $0 < \ell < L/Z$. When we add shortcuts, we expect that the distribution will be shifted to shorter path lengths.

Write the following three functions to find and analyze the path length distribution.

i **FindPathLengthsFromNode(graph, node)**, which returns for each *node*2 in the graph the shortest distance from *node* to *node*2. An efficient algorithm is a breadth-first traversal of the graph, working outward from node in shells. There will be a *currentShell* of nodes whose distance will be set to ℓ unless they have already been visited, and a *nextShell* which will be considered after the current one is finished (looking sideways before forward, breadth first), as follows.

- Initialize $\ell = 0$, the distance from *node* to itself to zero, and *currentShell* = [*node*].
- While there are nodes in the new *currentShell*:
  - start a new empty *nextShell*;
  - for each neighbor of each node in the current shell, if the distance to neighbor has not been set, add the node to *nextShell* and set the distance to $\ell + 1$;
  - add one to ℓ, and set the current shell to *nextShell*.
- Return the distances.

This will sweep outward from *node*, measuring the shortest distance to every other node in the network. (Hint: Check your code with a network with small N and small p, comparing a few paths to calculations by hand from the graph image generated as in part (a).)

ii **FindAllPathLengths(graph)**, which generates a list of all lengths (one per pair of nodes in the graph) by repeatedly using FindPathLengthsFromNode. Check your function by testing that the histogram of path lengths at $p = 0$ is constant for $0 < \ell < L/Z$, as advertised. Generate graphs at $L = 1,000$ and $Z = 2$ for $p = 0.02$ and $p = 0.2$; display the circle graphs and plot the histogram of path lengths. Zoom in on the histogram; how much does it change with p ? What value of p would you need to get "six degrees of separation"?

iii **FindAveragePathLength(graph)**, which computes the mean $\langle \ell \rangle$ over all pairs of nodes. Compute ℓ for $Z = 2, L = 100$, and $p = 0.1$ a few times; your answer should be around $\ell = 10$.

(c) Plotting average path length. Plot the average path length between nodes $\ell(p)$ divided by $\ell(p = 0)$ for $Z = 2, L = 50$, with p on a semi-log plot from $p = 0.001$ to $p = 1$. (Hint: Your curve should be similar to Fig. 2 of Watts and Strogatz [1], with the values of p shifted by a factor of 100.)

(d) Real network. Find a real network and calculate the mean distance and histogram of distances between its nodes.

In the small-world network, a few long edges result in a drastic reduction in the shortest paths between nodes in the network. It is often useful to measure how crucial a given node or edge is to these shortest paths. We say a node A is *between* two other nodes B and C if A is along a shortest path between B and C. In class, we used this to derive the *betweenness centrality* of a node. Pick three nodes in the real network you found above and find their betweenness centrality using Gephi (or the Python package NetworkX).

3. Network flow issues come up in dealing with natural disasters and other crises, since major unexpected events often require the movement and evacuation of large numbers of people in a short amount of time.

Consider the following scenario. Due to large-scale flooding in a region, paramedics have identified a set of n injured people distributed across the region who need to be rushed to hospitals. There are k hospitals in the region, and each of the n people needs to be brought to a hospital that is within a half-hour's driving time of their current location (so different people will have different options for hospitals, depending on where they are right now).

At the same time, one doesn't want to overload any one of the hospitals by sending too many patients its way. The paramedics are in touch by cell phone, and they want to collectively work out whether they can choose a hospital for each of the injured people in such a way that the load on the hospitals is balanced: Each hospital receives at most $\lceil n/k \rceil$ people.

Describe a network flow algorithm that takes the given information about the people's locations and determines whether this is possible. You do not need to write code for this problem, but you need to explain the algorithm clearly (see class slides for example).

4. Consider a large-scale atmospheric science experiment. You need to get good measurements on a set S of n different conditions in the atmosphere (such as the ozone level at various places), and you have a set of m balloons that you plan to send up to make these measurements. Each balloon can make at most two measurements. Unfortunately, not all balloons are capable of measuring all conditions, so for each balloon $i = 1, ..., m$, they have a set S_i of conditions that balloon i can measure. Finally, to make the results more reliable, you plan to take each measurement from at least k different balloons. (Note that a single balloon should not measure the same condition twice.)

Example. Suppose that $k = 2$, there are $n = 4$ conditions labeled $c1, c2, c3, c4$, and there are m =4 balloons that can measure conditions, subject to the limitation that $S1 = S2 = \{c1, c2, c3\}$, and $S3 = S4 = \{c1, c3, c4\}$. Then one possible way to make sure that each condition is measured at least $k = 2$ times is to have

- balloon 1 measure conditions $c1, c2$,

- balloon 2 measure conditions $c2, c3$,
- balloon 3 measure conditions $c3, c4$, and
- balloon 4 measure conditions $c1, c4$.

Describe a network flow algorithm that takes the input to an instance of this problem (the n conditions, the sets S_i for each of the m balloons, and the parameter k) and decides whether there is a way to measure each condition by k different balloons, while each balloon only measures at most two conditions. You do not need to write code for this problem, but you need to explain the algorithm clearly (see class slides for example).

References

[1] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, no. 6684, pp. 440–442, 1998.